

LC 229 Majority element-II

Approach:

- Extended version of yesterdays LC169 Majority Element but this time instead of one , we will initialize 2 element and 2 count variables.
- Why 2? It is impossible when the required condition is greater than $n/3$, hence if there are 3 or more then the condition fails($n/3+n/3+n/3=n$)
- Store first element in element 1 and increment count1 by 1 . Similarly same with element 2 and count2+=1
- If count1 and count 2 are both 0, we make them 1 and assign the first element of the array to their respective variables
- Now we will create an empty set(why set? To handle duplicate results)
- And now we will add the elements into the set when they satisfy the given condition i.e., greater than $n/3$
- Lastly return the list version of the set

LC 62 Unique Paths

Approach:

- This Grid DP can be solved using memorization, tabulation and space optimized DP.
- Create a function where initialize 2 arrays one previous and one temporary
- Each cell can be reached from above/up($dp[i-1][j]$) and left($dp[i][j-1]$)
- Base case , make the first cell $dp[0][0]=1$
- Use both arrays to compute the value of $dp[i][j]$
- Return $prev[-1]$ as the answer, meaning reaching the end point having unique paths

LC 493 Reverse Pairs

Approach:

- Using Merge sort to recursively divide the problem into two equal halves
- Create countpair function to count the number of pairs after the array is recursively sorted
- The we merge the whole divided elements into a temporary array then add the elements of the temporary array back to the original array
- Cast 'long' to the element to avoid integer overflow(if using java)